

Islamic University of Technology

Department of Computer Science and Engineering (CSE)

B.Sc. in Software Engineering

SWE 4304: Software Project Lab II

Summer, 2023-2024

Software Design Report

Plastic_box_solver

(Software Design Diagrams & Software Requirements Specification)

Group 6

S. M. Tahsinuzzaman Emon, 230042104

S. M. Samiul Hossain, 230042110

Md. Muntahi Hasan Akhiar, 230042118

Supervisor

Aashnan Rahman

Junior Lecturer, CSE

Date: July 27, 2026

Contents

1 Introduction **2**

2 Design Approach and Justification **2**

3 Functional Decomposition Diagram **3**

4 Data Flow Diagram (DFD) **4**
4.1 Level 0 (Context) and Level 1 4
4.2 Level 2: Explosion of Process 2.0 (Capture & Recognize Cube) 6
4.3 Level 2: Explosion of Processes 3.0 & 4.0 (Render 3D + Compute Solution) 7

5 Three-Layer Architecture Diagram **8**

6 Class Diagram **9**

7 Software Requirements Specification (SRS) **10**
7.1 Project Overview 10
7.2 Functional Requirements 10
7.3 Non-Functional Requirements 21

1 Introduction

This report presents the software design of **Plastic_box_solver**, a browser-based Rubik's Cube scanning and solving assistant, and attaches the project's detailed Software Requirements Specification (SRS). Because the system does not follow a single programming paradigm: it combines a component-based React front end, a stateless OpenCV scan pipeline and stateless solving algorithms with an object-oriented data layer: the design is documented with both structured diagrams (a Functional Decomposition Diagram and a multi-level Data Flow Diagram) and object-oriented diagrams (a three-layer Architecture Diagram and a Class Diagram). Each diagram is followed by a short explanation.

2 Design Approach and Justification

Plastic_box_solver is a full-stack web application whose components do not all follow one paradigm, and the diagrams reflect that honestly. The **front end** (React with React Three Fiber and Zustand) is component- and function-based; the **scan pipeline** (OpenCV colour extraction and validation) and the **solving algorithms** (Kociemba two-phase and CFOP) are stateless transformations that are most naturally expressed as functions rather than classes. Only the **data / persistence layer** is inherently object-oriented, since the SQLAlchemy models (User, SolveSession, Badge, LeaderboardEntry) are classes.

The system is therefore presented as **primarily structured/functional with an object-oriented data layer**. The Functional Decomposition Diagram captures *what* the system does, the Data Flow Diagram captures *how data moves*, the Architecture Diagram captures *how the code is organised*, and the Class Diagram models the object-oriented data layer.

3 Functional Decomposition Diagram



Figure 1: Functional decomposition of Plastic_box_solver.

The Functional Decomposition Diagram breaks the system top-down into its major functional areas and then into the concrete sub-functions under each, answering *what* the system does independently of data flow or code structure. The root node branches into six functional areas, and each area branches further into its own sub-functions, giving a strict hierarchy rather than a sequence of steps. The six areas are:

1. **User Account Management**: registration, login/logout, profile management and password reset.
2. **Cube Input & Recognition**: camera scan or manual input, colour extraction, auto-calibration, and validation with a 2D preview.
3. **Cube Visualization**: 3D rendering, orbit/rotate view, and face rotation.
4. **Solving Engine**: auto-solve, guided solve, and scramble generation.
5. **Solve Session**: interactive (timed) solving, guided step-by-step solving, auto-solve animation, and efficiency calculation.

6. **Statistics & Persistence**: time/move tracking, history and personal best, leaderboard, and badges/sharing.

These areas map directly onto the functional requirements in Section 7 and form a natural checklist for implementation and testing.

4 Data Flow Diagram (DFD)

The diagrams in this section are **logical** Data Flow Diagrams: they describe *what* data moves between *what* functions, independently of the technology used to implement them. They do not show physical detail such as servers, HTTP endpoints, file formats or specific database tables; that physical view is given instead by the Architecture Diagram in Section 5.

The notation used in all three DFD levels is summarised below and is also reproduced as a legend inside Figures 2 and 3:

- **External entity** (plain rectangle): a person or system outside the software boundary that supplies or receives data (here, the *User*).
- **Process** (rounded box, numbered *n.0*): a transformation that takes input data and produces different output data.
- **Data store** (open-ended box, labelled *D1–D3*): data at rest between processes.
- **Data flow** (solid labelled arrow): a specific item of data moving in the direction of the arrow.
- **User / session ID** (dashed labelled arrow): the authenticated user identity propagated to the per-user processes; drawn dashed only to distinguish it visually from ordinary data.

4.1 Level 0 (Context) and Level 1

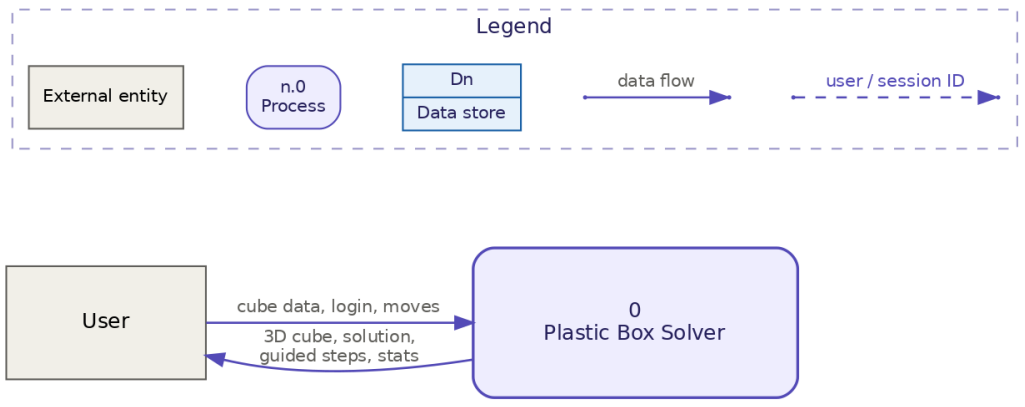


Figure 2: Level 0 (context) data flow diagram.

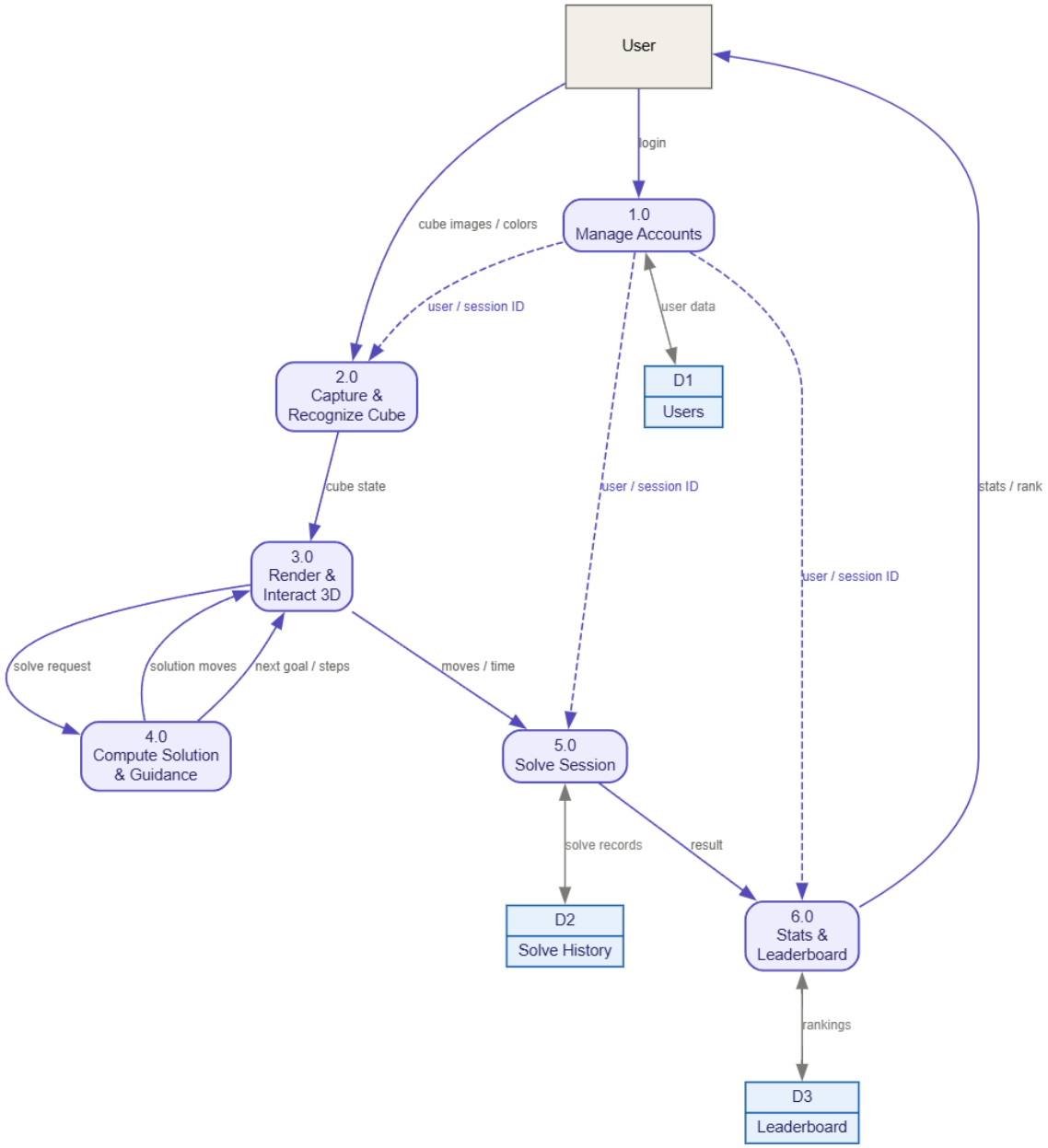


Figure 3: Level 1 data flow diagram.

Level 0 (the context diagram) fixes the system boundary: the User exchanges data with the system as a single box.

Level 1 explodes that box into six processes and the three PostgreSQL stores they use. A full solve can be traced through the arrows as a step-by-step walk-through:

1. **1.0 Manage Accounts**: After login, issues a *user / session ID* (dashed) that flows into the per-user processes so scans, solves and stats are attributed to the right account.
2. **2.0 Capture & Recognize**: Cube images enter from the user and become a validated cube state passed to **3.0 Render & Interact 3D**.

3. **3.0 Render & Interact 3D**: Receives the validated cube state to render the interactive 3D cube and displays the solver's result and tutorial hints back to the user.
4. **4.0 Compute Solution & Guidance**: Returns the *solution moves and* the *next-goal guided steps* back to the renderer so the user sees the result and the tutorial hints. Note that **4.0** touches no data store: the solver is a pure computation.
5. **5.0 Solve Session**: Records the run in **D2 Solve History**.
6. **6.0 Stats & Leaderboard**: Turns that history into a ranking in **D3**.

4.2 Level 2: Explosion of Process 2.0 (Capture & Recognize Cube)

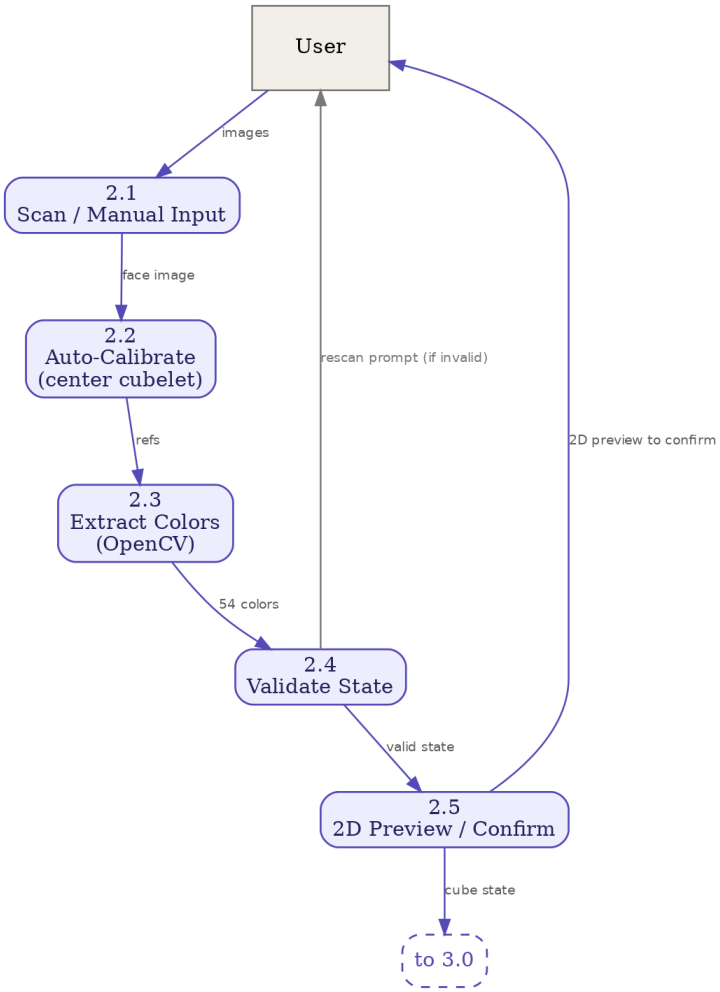


Figure 4: Level 2 DFD: internal processes of Capture & Recognize Cube.

This Level 2 diagram explodes process **2.0** into its internal steps:

1. **2.1 Receive Face Images**: Receives the face images from the camera scan or manual input.
2. **2.2 Auto-Calibrate Colors**: Auto-calibrates using the centre cubelet as the reference colour.

3. **2.3 Extract Sticker Colors**: Extracts the 54 sticker colours with OpenCV.
4. **2.4 Validate Cube State**: Validates that the colours form a legal, solvable cube (prompting a rescan if not).
5. **2.5 Confirm 2D Preview**: Shows a 2D preview for the user to confirm before the validated cube state is passed on to process 3.0.

4.3 Level 2: Explosion of Processes 3.0 & 4.0 (Render 3D + Compute Solution)

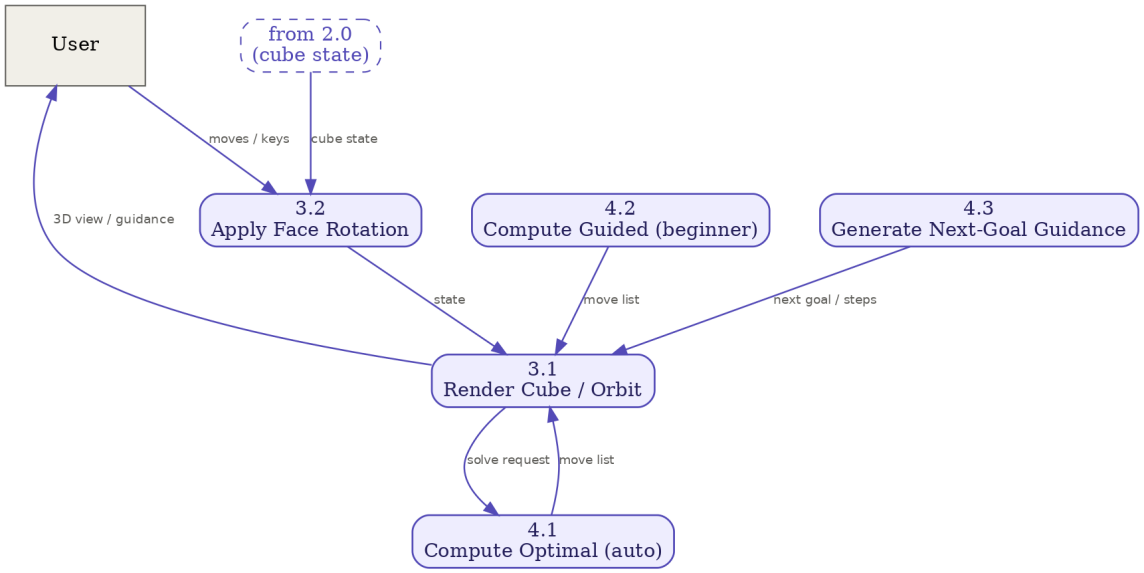


Figure 5: Level 2 DFD: internal processes of Render 3D and Compute Solution.

This Level 2 diagram explodes the render-and-solve loop into a five-step interaction:

1. **3.1 Render & Orbit Cube**: Renders and orbits the 3D cube view for the user.
2. **3.2 Apply Face Rotation**: Applies the user's own manual face rotations to the cube state.
3. **4.1 Optimal Solver (Auto-Solve)**: On request, computes an optimal solution and returns a *move list* to the renderer.
4. **4.2 Guided Solver**: Computes a beginner-friendly guided solution and returns its *move list* to the renderer.
5. **4.3 Tutorial Generator**: Generates the *next goal and step instructions* for the tutorial, which the renderer displays to the user alongside the 3D view.

5 Three-Layer Architecture Diagram

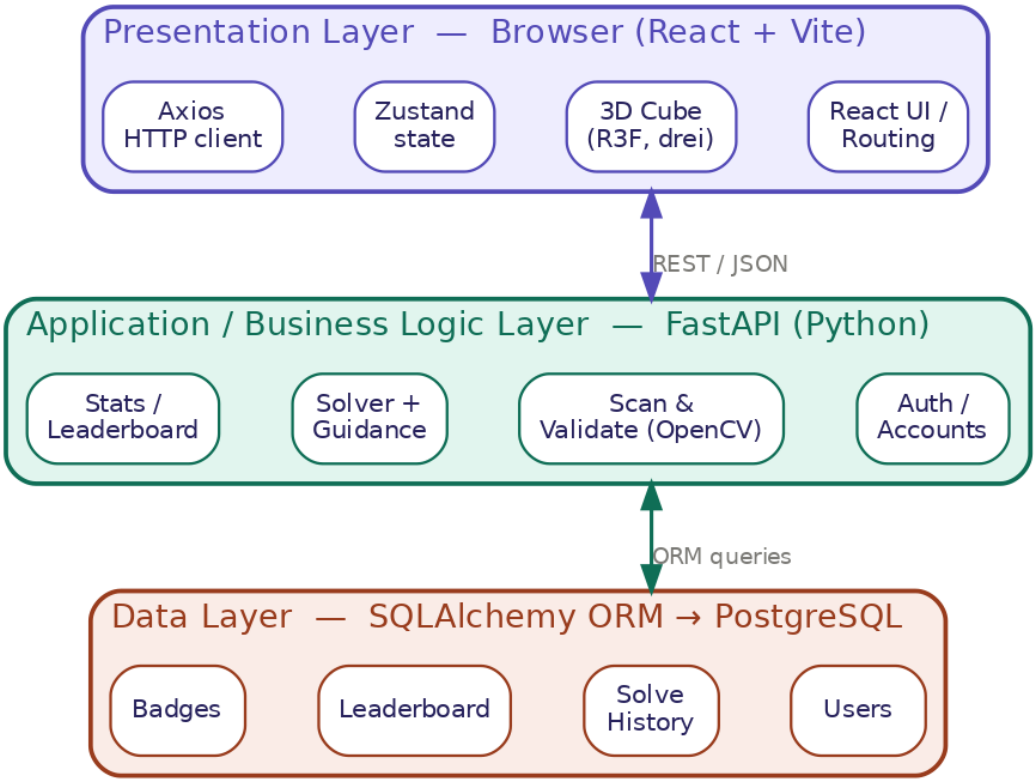


Figure 6: Three-layer software architecture.

The Architecture Diagram organises the code into three layers with a strict top-to-bottom dependency:

- **Presentation Layer (Client):** Runs in the browser using React + Vite, a React Three Fiber interactive 3D cube, Zustand for state management, and an Axios HTTP client.
- **Application / Business Logic Layer (Server):** Built with the FastAPI backend, grouping server-side operations into authentication, the OpenCV scan-and-validate pipeline, the solver-plus-guidance module, and stats/leaderboard logic.
- **Data Layer (Persistence):** Persists all application state through the SQLAlchemy ORM into PostgreSQL.
- **Strict Layer Separation:** Each layer talks only to the one directly below it—the presentation layer communicates over REST/JSON, and the application layer uses ORM queries. This keeps the UI, business rules, and storage independently testable and replaceable.

6 Class Diagram

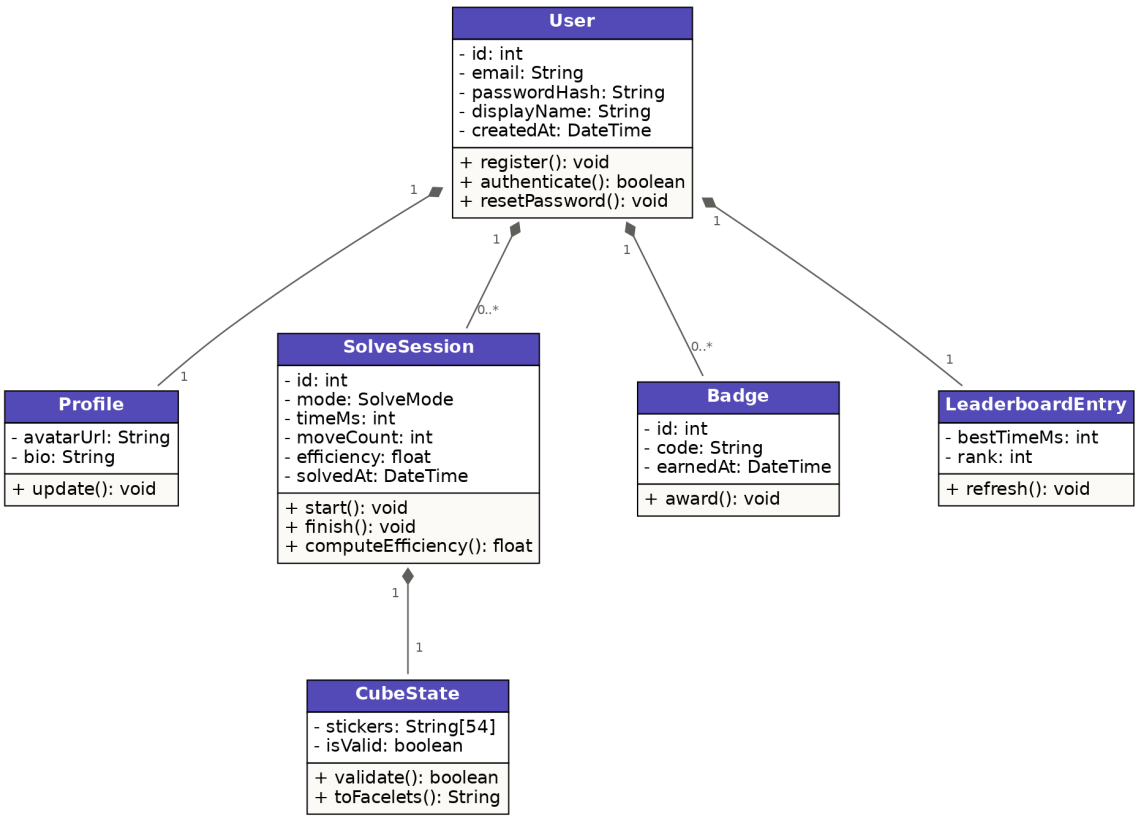


Figure 7: Class diagram of the object-oriented data / domain layer.

The Class Diagram models the object-oriented persistent domain classes managed by SQLAlchemy using standard UML notation:

- **UML Class Boxes:** Each class is drawn as a three-compartment box containing the class name, attributes (*visibility name: Type*), and operations (*visibility name(): ReturnType*), where '−' denotes private and '+' denotes public visibility.
- **Composition Relationships:** All associations in this model are **compositions** (solid line with a filled diamond at the owning end):
  - **User** composes exactly one **Profile** (`1--1`) and one **LeaderboardEntry** (`1--1`).
  - **User** composes zero or more **SolveSession** and **Badge** records (`1--0..*`).
  - **SolveSession** composes exactly one **CubeState** (`1--1`), representing the scanned configuration it was solved from.
- **Lifecycle Ownership:** Composition is used because child objects depend entirely on their parent's existence; deleting a user automatically deletes that user's profile, badges, and solve records.
- **Foreign Keys Omitted:** Foreign-key fields (e.g., `userId`) are deliberately omitted from class attributes. In UML, the association line itself represents the link, avoiding duplication. Foreign keys appear only in the physical database schema.

7 Software Requirements Specification (SRS)

The complete Software Requirements Specification (SRS) spreadsheet.

7.1 Project Overview

A browser-based Rubik's Cube scanning and solving assistant. Users scan or manually input their physical cube's state, the system validates and renders it as an interactive 3D model, then computes a solution (optimal via the Kociemba two-phase algorithm, or beginner's-method for guided learning). Users can solve interactively with a live timer, follow guided step-by-step instructions, or watch an automatic solve animation, with results (time, move count, efficiency) tracked, saved to history, and ranked on a global leaderboard.

Field	Value
Group ID	6
Student IDs	230042104, 230042110, 230042118
Title	Plastic Box Solver

7.2 Functional Requirements

The system defines 29 functional requirements (FR-01 … FR-22). Each is listed below with its priority and current status, the requirement statement, its user story, and its acceptance criteria.

FR-01: New User Registration

Priority: Medium • *Status:* To Do
Requirement. The system shall allow a new user to register for an account using email and password.
User story. As a first-time visitor to the platform, I want to create an account using my email address and a password, so that I can save my scan history, solve times, and leaderboard rank across sessions.
Acceptance criteria:

- Given a visitor enters a valid, unused email and a password of at least 8 characters, when they submit the registration form, then an account is created and the user is redirected to the Main Menu.
- Given a visitor enters an email that already exists in the system, when they submit the form, then the system displays 'Email already registered' and does not create a duplicate account.
- Given a visitor enters a password shorter than 8 characters, when they submit the form, then the system blocks submission and shows an inline validation error.
- Given a registration succeeds, when the password is persisted, then it is stored only in hashed form, never in plaintext.

FR-02: Log In to Existing Account

Priority: Medium • *Status:* To Do
Requirement. The system shall allow a registered user to log in using their credentials.

User story. As a registered user returning to the app, I want to log in with my email and password, so that I can resume practicing without losing my saved history or leaderboard standing.

Acceptance criteria:

- Given a registered user enters correct credentials, when they submit the login form, then they are authenticated and taken to the Main Menu.
- Given a user is successfully authenticated, when the session is created, then it persists across page reloads until logout or expiry.
- Given an unauthenticated user tries to access a protected page directly by URL, when the page loads, then they are redirected to the login screen.

FR-03: Manage Personal Profile

Priority: Low • *Status:* To Do

Requirement. The system shall allow a logged-in user to view and update their personal profile.

User story. As a logged-in user, I want to view and edit my profile details such as display name and avatar, so that my identity is shown correctly on my solve history and the leaderboard.

Acceptance criteria:

- Given a user opens the profile page, when it loads, then their current display name and email are shown.
- Given a user changes their display name to a valid, unique value, when they save, then the new name is reflected immediately on the leaderboard.
- Given a user attempts to set a display name already taken by another user, when they save, then the system rejects the change with an explanatory error.
- Given a user updates their profile, when the update completes, then a success confirmation is shown.

FR-04: Scan Physical Cube with Camera

Priority: High • *Status:* In Progress

Requirement. The system shall accept a scrambled cube state captured via the device camera.

User story. As a user who owns a physical Rubik's Cube, I want to scan each of the six faces using my phone or webcam camera, so that I don't have to manually enter every sticker color by hand.

Acceptance criteria:

- Given the user grants camera permission, when they align a cube face inside the on-screen guide frame, then the system captures the image for that face.
- Given all six faces have been captured, when the last face is submitted, then the backend receives all six images for color processing.
- Given camera permission is denied, when the user opens the scan screen, then the system shows an explanatory message and offers the manual input option.
- Given a captured image is blurry or too dark to process, when it is submitted, then the system prompts the user to retake that specific face.

FR-05: Manually Enter Cube Colors

Priority: Medium • *Status:* To Do
Requirement. The system shall provide a manual click-input interface for setting sticker colors when camera scanning is unreliable.
User story. As a user in poor lighting conditions, I want to set each of the 54 sticker colors by clicking on a flat 2D cube net, so that I can still submit an accurate cube state when camera scanning fails.
Acceptance criteria:

- Given the manual input screen is open, when the user clicks a sticker cell, then a color palette appears for selecting that sticker's color.
- Given the user has not yet assigned a color to every one of the 54 stickers, when they attempt to submit, then the system blocks submission and highlights unfilled stickers.
- Given all 54 stickers are assigned, when the user submits, then the cube state is sent for validation using the same rules as camera-scanned input.
- Given the user wants to start over, when they click 'Reset', then all 54 stickers are cleared back to an unset state.

FR-06a: Extract Sticker Colors from Scanned Images

Priority: High • *Status:* In Progress
Requirement. The system shall detect and extract all 54 sticker colors from the six submitted face images.
User story. As a user who just finished scanning their cube, I want the system to automatically extract all 54 sticker colors from my six submitted face images, so that I don't have to manually identify each sticker color myself.
Acceptance criteria:

- Given six submitted face images, when OpenCV processes them, then exactly nine colors per face are extracted and mapped to the 54 sticker positions.

FR-06b: Validate Scanned Cube State Legality

Priority: High • *Status:* In Progress
Requirement. The system shall validate that the extracted 54-color cube state is legal and solvable, rejecting invalid configurations with specific feedback.
User story. As a user who just finished scanning their cube, I want the system to check that my scanned colors form a legal, solvable cube, so that I find out immediately if I need to rescan a face instead of getting a failed solve later.
Acceptance criteria:

- Given the extracted 54-color state has an illegal color distribution (e.g. more than 9 stickers of one color), when validation runs, then the system rejects the state and tells the user which face(s) to rescan.
- Given the extracted state has a legal color count but an unsolvable permutation or orientation, when validation runs, then the system rejects it with a message explaining the cube appears physically inconsistent.

FR-06c: Confirm Cube State via 2D Preview

Priority: High • *Status:* In Progress

Requirement. The system shall display a 2D preview of the validated cube state for user confirmation before solving begins.
User story. As a user whose scanned cube state has passed validation, I want to see a 2D preview of my confirmed cube state, so that I can visually verify the scan is accurate before proceeding to solve.
Acceptance criteria:

- Given the extracted 54-color state passes all legality checks, when validation completes, then the user is shown a 2D preview for final confirmation before solving begins.

FR-07: View and Rotate Interactive 3D Cube

Priority: High • *Status:* In Progress
Requirement. The system shall render the confirmed cube state as a fully interactive, rotatable 3D model in the browser.
User story. As a user reviewing their scanned cube, I want to freely rotate and inspect a 3D model that mirrors my physical cube exactly, so that I can verify the scan is correct and get oriented before I start solving.
Acceptance criteria:

- Given a validated cube state, when the 3D view loads, then all 26 cubies render with colors matching the confirmed 2D preview.
- Given the user drags on the 3D view, when they release, then the camera orbits smoothly around the cube without lag on both desktop and mobile Chromium browsers.
- Given the user triggers a face rotation via an on-screen button or swipe or keyboard input, when the animation plays, then the visual state and the underlying logical cube state stay in sync.
- Given the 3D view is displayed on a screen as narrow as 360px, when it renders, then all controls remain usable without overlapping elements.

FR-08a: Generate Optimal Solving Algorithm (Auto-Solve)

Priority: High • *Status:* In Progress
Requirement. The system shall compute an optimal solution using the Kociemba two-phase algorithm for Auto-Solve mode.
User story. As a user who has confirmed their scanned cube state and selected Auto-Solve, I want the system to calculate an optimal solving sequence using the Kociemba two-phase algorithm, so that I can watch my exact cube get solved automatically with minimal moves.
Acceptance criteria:

- Given a validated, solvable cube state, when the user requests Auto-Solve, then the Kociemba two-phase algorithm returns a move sequence within 2 seconds.
- Given the solver receives an unsolvable or malformed state, when it attempts to compute, then it returns a clear error instead of crashing or hanging.
- Given a solution is computed, when it is returned to the frontend, then it includes the total move count for later efficiency comparison.

FR-08b: Generate Guided Solving Algorithm

Priority: High • *Status:* In Progress

Requirement. The system shall compute a beginner's-method solution for Guided Mode.
User story. As a user who has confirmed their scanned cube state and selected Guided Mode, I want the system to calculate a beginner's-method move list for my exact cube, so that I can follow it as a step-by-step guide instead of an optimal solution.
Acceptance criteria:

- Given the user selects Guided Mode, when a solution is requested, then the system returns a beginner's-algorithm-based move list instead of a raw optimal solution.
- Given the solver receives an unsolvable or malformed state, when it attempts to compute, then it returns a clear error instead of crashing or hanging.
- Given a solution is computed, when it is returned to the frontend, then it includes the total move count for later efficiency comparison.

FR-09: Solve the Cube Myself with Live Timer

Priority: Medium • *Status:* To Do
Requirement. The system shall support an Interactive Solve mode where the user manipulates the 3D cube themselves.
User story. As a user practicing their own solving method, I want to make my own moves on the 3D cube while a timer and move counter run live, so that I can measure my solving speed and efficiency without any hints.
Acceptance criteria:

- Given the user enters Interactive Mode, when they make the first move, then the timer starts automatically and the move counter increments with every subsequent move.
- Given the user reaches a fully solved state, when the last move is made, then the timer stops immediately and the final time and move count are recorded.
- Given a solve is completed, when results are displayed, then the optimal move count from FR-08a is shown alongside the user's move count for comparison.
- Given the user wants to abandon the attempt, when they click 'Reset Cube', then the timer, move counter, and cube state all reset to the original scrambled state.

FR-10: Follow Step-by-Step Guided Instructions

Priority: Medium • *Status:* To Do
Requirement. The system shall support a Guided Solve mode that presents move-by-move instructions following traditional beginner's algorithm.
User story. As a user learning to solve the cube for the first time, I want to see one move at a time in a sidebar and apply it on my physical cube before advancing, so that I can learn beginner's algorithms without switching attention between a tutorial video and my cube.
Acceptance criteria:

- Given Guided Mode is active, when the beginner's-algorithm move list loads, then the current move is visually highlighted in the sidebar and mirrored on the 3D model.
- Given the user clicks 'Next', when the current move is applied, then the 3D cube animates that single move and advances the highlight to the following move.
- Given the user clicks 'Previous', when triggered, then the last applied move is undone on the 3D model and the highlight moves back one step.

- Given the user reaches the final move in the list, when it is applied, then the system confirms the cube is fully solved and offers to return to the Main Menu.

FR-11: Watch the Cube Solve Itself Automatically

Priority: Medium • *Status:* To Do
Requirement. The system shall support an Auto-Solve mode that animates the full computed solution sequence without user input.
User story. As a user who wants to see the solution demonstrated, I want to trigger Auto-Solve and watch the entire move sequence animate on its own, so that I can study the solving technique passively before attempting it myself.
Acceptance criteria:

- Given the user selects Auto-Solve, when they click 'Play', then the full move sequence animates automatically at a consistent, smooth frame rate.
- Given the animation is playing, when the user clicks 'Pause', then the animation halts on the current move and can be resumed from the same point.
- Given the user wants to review a specific moment, when they drag the progress scrubber, then the 3D cube jumps to the corresponding move in the sequence.
- Given the animation reaches the last move, when it completes, then the cube visually matches a fully solved state and playback controls reset.
- Given the user is watching an auto solve, when they adjust the speed (slider/button), then the animation speed changes accordingly without requiring them to click next.

FR-12a: Track Solve Time and Move Count

Priority: Low • *Status:* To Do
Requirement. The system shall track solve time and move count per session in Interactive Mode.
User story. As a user practicing in Interactive Mode, I want my solve time and move count to be tracked automatically during my attempt, so that I have accurate raw data recorded for every attempt without manual timing.
Acceptance criteria:

- Given a completed solve, when results are calculated, then the recorded time and move count are stored for that session.
- Given results are shown, when the user clicks 'Save', then the session record is persisted to their solve history.

FR-12b: View Solve Efficiency Percentage

Priority: Low • *Status:* To Do
Requirement. The system shall calculate an efficiency percentage comparing the user's move count to the system-generated solution.
User story. As a user who just completed a solve in Interactive Mode, I want to see my efficiency percentage compared to the system-generated solution, so that I know exactly how much room for improvement I have.
Acceptance criteria:

- Given a completed solve, when results are calculated, then efficiency is computed as system-generated move count divided by the user's move count, expressed as a percentage.

- Given the calculated efficiency, when it is displayed, then it is shown alongside raw solve time and move count in the results panel.
- Given the user's move count is lower than or equal to the system-generated count, when efficiency is computed, then it is capped at 100% rather than exceeding it.

FR-13a: Persist Solve History

Priority: Low • *Status:* To Do
Requirement. The system shall persist user solve history server-side.
User story. As a user tracking long-term progress, I want every completed solve to be saved to my account on the server, so that my history is available no matter which device I log in from.
Acceptance criteria:

- Given a user has completed at least one saved solve, when they open the History page, then every past solve is listed with date, time, move count, and efficiency.
- Given a user has solved on both desktop and mobile, when they log in from either device, then their complete history appears identically since it is stored server-side.
- Given a user has no completed solves yet, when they open the History page, then the system shows an empty-state message instead of an error.

FR-13b: View Personal Best

Priority: Low • *Status:* To Do
Requirement. The system shall identify and display the user's personal best solve from their history.
User story. As a user tracking long-term progress, I want my fastest recorded solve to be flagged as my Personal Best, so that I can quickly see the benchmark I'm trying to beat.
Acceptance criteria:

- Given a user's history contains multiple solves, when the page loads, then the single fastest solve is flagged as the user's Personal Best.

FR-14: Compare My Rank on the Global Leaderboard

Priority: Low • *Status:* To Do
Requirement. The system shall display a global leaderboard ranking users by solve performance.
User story. As a competitive user comparing progress against others, I want to view a global leaderboard ranked by solve time and efficiency, with my own rank highlighted, so that I stay motivated to practice and can see how I measure up against other users.
Acceptance criteria:

- Given multiple users have saved solves, when the leaderboard loads, then entries are sorted by best solve time by default, with ties broken by higher efficiency.
- Given the logged-in user appears anywhere on the leaderboard, when it renders, then their own row is visually highlighted even if they must scroll to see it.
- Given a new personal best is saved, when the leaderboard is next viewed, then it reflects the updated ranking without requiring more than one manual page refresh.
- Given the leaderboard has more entries than fit on one screen, when the user scrolls or paginates, then additional ranked entries load correctly in order.

FR-15: Reset Forgotten Password

Priority: Medium • *Status:* To Do
Requirement. The system shall allow a registered user to reset a forgotten password via an emailed reset link.
User story. As a registered user who has forgotten their password, I want to request a password reset link sent to my registered email, so that I can regain access to my account without contacting support.
Acceptance criteria:

- Given a user enters a registered email on the 'Forgot Password' screen, when they submit it, then a time-limited reset link is emailed to that address.
- Given a user enters an email not associated with any account, when they submit it, then the system shows a generic confirmation message without revealing whether the account exists.
- Given a user clicks a valid, unexpired reset link, when they submit a new password of at least 8 characters, then their password is updated and they are redirected to the login screen.
- Given a reset link is older than 30 minutes or has already been used once, when the user opens it, then the system rejects it and prompts them to request a new link.

FR-16: Generate Practice Scramble

Priority: Low • *Status:* To Do
Requirement. The system shall generate a randomized, solvable scramble sequence for practice sessions.
User story. As a user who wants to practice without a physical mixed-up cube, I want to click 'New Scramble' and have the 3D cube randomize into a fresh, solvable state, so that I can start a new practice attempt immediately without manually mixing my physical cube.
Acceptance criteria:

- Given the user clicks 'New Scramble', when the request is processed, then the system generates a sequence of at least 20 random face turns and applies it to the 3D cube.
- Given a scramble is generated, when it is applied, then the resulting cube state is guaranteed solvable, matching the same legality rules used for camera-scanned states.
- Given a new scramble is requested, when it is generated, then the exact scramble sequence is displayed to the user so they can optionally replicate it on a physical cube.
- Given a scramble has just been applied, when the user starts Interactive Mode, then the timer and move counter begin from zero using this new scrambled state as the baseline.

FR-17: Auto-Calibrate Color via Center Cubelet

Priority: High • *Status:* In Progress
Requirement. The system shall automatically calibrate color detection for each face by reading the center cubelet as the reference color.
User story. As a user scanning a cube under varying lighting conditions, I want the system to automatically determine the base color of each face from its center cubelet, so

that I don't have to manually calibrate colors or worry about lighting differences across faces.

Acceptance criteria:

- Given a user scans a face, when the image is processed, then the system identifies the color of the center cubelet and uses it as the reference for that face's color.
- Given the center cubelet color is determined, when the other 8 cubelets are evaluated, then their colors are classified relative to the 6 reference center colors.
- Given the system relies on center cubelets, when no manual calibration is provided, then the system still accurately identifies colors despite minor lighting variations.

FR-18: Undo and Redo Moves During Interactive Solve

Priority: Low • *Status:* To Do

Requirement. The system shall allow a user to undo and redo individual moves during an Interactive Mode solve.

User story. As a user practicing their own solving method in Interactive Mode, I want to undo my last move or redo a move I just undid, so that I can correct an accidental turn without resetting my entire attempt.

Acceptance criteria:

- Given at least one move has been made in the current attempt, when the user clicks 'Undo', then the 3D cube reverts to the state before that move and the move counter decrements by one.
- Given a move has just been undone, when the user clicks 'Redo', then the undone move is reapplied and the move counter increments back.
- Given the user makes a brand-new move after undoing one or more moves, when that new move is applied, then the previously undone redo history is cleared.
- Given undo or redo is used, when the timer is running, then the elapsed time continues uninterrupted and is not affected by the undo/redo action.

FR-19: Share Solve Result

Priority: Low • *Status:* To Do

Requirement. The system shall let a user export or share a summary of a completed solve, including time, move count, and efficiency.

User story. As a user who just completed a solve they are proud of, I want to generate a shareable summary card of my time, move count, and efficiency, so that I can post or send my result outside the platform without manually re-typing the numbers.

Acceptance criteria:

- Given a completed and automatically saved solve, when the user clicks 'Share Result', then the system generates an image or link summarizing time, move count, efficiency, and solve mode.
- Given a share link is generated, when it is opened by anyone, then it displays a read-only summary without exposing the viewer's own account data.
- Given the user clicks 'Copy Link' or 'Download Image', when the action completes, then the system confirms success with a brief on-screen message.

FR-20: Control Cube with Keyboard Shortcuts

Priority: Medium • *Status:* To Do

Requirement. The system shall allow a user to perform cube face rotations using keyboard shortcuts, mapping standard cube notation keys (U, D, L, R, F, B) to clockwise turns and their Shift-modified versions to counter-clockwise turns.
User story. As a user practicing their own solving method in Interactive Mode, I want to press notation keys like R, U, L, D, F, B (and Shift+key for the reverse direction) to rotate cube faces instead of clicking, so that I can turn the cube as fast as I would with a physical speedcube, without breaking my flow to reach for a mouse.
Acceptance criteria:

- Given Interactive Mode is active and the cube has focus, when the user presses R, then the right face rotates 90° clockwise and the move is logged in the move counter as "R".
- Given Interactive Mode is active, when the user presses Shift+R, then the right face rotates 90° counter-clockwise and the move is logged as "R'".
- Given the user presses any of U, D, L, R, F, B (with or without Shift), when the key is pressed, then the corresponding face rotates and the on-screen 3D model animates the turn within the same frame budget as a button-triggered move.
- Given the user holds a key down or presses keys faster than the current animation can complete, when additional keypresses are received, then moves are queued and executed in order rather than dropped or executed out of sequence.
- Given the user is not in Interactive Mode (e.g. viewing Auto-Solve or Guided Mode), when a notation key is pressed, then no rotation occurs and no move is logged.
- Given the user is typing in a text field (e.g. display name, email) elsewhere on the page, when a notation key is pressed, then it is treated as normal text input and does not trigger a cube rotation.
- Given a double-layer or wide-move variant is not supported in this version, when the user presses a key not mapped to a face (e.g. M, E, S), then the system ignores it without error.

FR-21a: Earn Milestone Achievement Badge

Priority: Low • *Status:* To Do
Requirement. The system shall detect when a user crosses a predefined milestone threshold (cumulative solve count or individual solve time) and award the corresponding badge.
User story. As a returning user tracking long-term progress, I want to automatically receive a badge when I hit a milestone, such as completing my 10th solve or finishing a solve in under 100 seconds, so that I stay motivated by visible markers of progress instead of only comparing raw numbers.
Acceptance criteria:

- Given a user completes and saves their 10th total solve, when the save is processed, then the "10 Solves" badge is awarded to their account and a congratulatory notification is shown.
- Given a user completes and saves a solve with a recorded time under 100 seconds, when the save is processed, then the "Sub-100" badge is awarded if the user does not already have it.
- Given a user has already earned a specific badge, when they complete another solve meeting that same milestone's criteria again, then the badge is not duplicated and no repeat notification is shown.

- Given multiple milestone thresholds are crossed by a single solve (e.g. it is simultaneously the user's 50th solve and their first sub-60-second solve), when the solve is saved, then all qualifying badges are awarded in the same transaction.
- Given a milestone check is running, when it evaluates solve count or time thresholds, then it reads only the requesting user's own solve history, never another user's data.

FR-21b: View Earned Badges on Profile

Priority: Low • *Status:* To Do
Requirement. The system shall display all badges earned by a user on their profile, ordered by date unlocked.
User story. As a user who has earned one or more badges, I want to see all my earned badges on my profile, in the order I unlocked them, so that I can visibly track my progression over time.
Acceptance criteria:

- Given a user views their profile, when the page loads, then all earned badges are displayed in the order they were unlocked, each with the date earned.

FR-21c: Retroactive Badge Evaluation on New Threshold

Priority: Low • *Status:* To Do
Requirement. The system shall retroactively evaluate existing users' historical solves when an admin adds a new milestone threshold.
User story. As an admin adding a new milestone threshold, I want existing users' historical solves to be automatically re-evaluated against the new threshold, so that already-qualifying users receive the new badge without needing to complete a new solve.
Acceptance criteria:

- Given the system defines milestone thresholds (e.g. 10, 50, 100, 500 solves; sub-100s, sub-60s, sub-30s times), when a new threshold is added by an admin, then existing users' historical solves are retroactively evaluated so already-qualifying users receive the new badge without needing a new solve.

FR-22: Log Out of Account

Priority: Low • *Status:* To Do
Requirement. The system shall allow a user to log out of their account from any page.
User story. As a logged-in user, I want to log out of my account, so that I can end my session and protect my account on shared devices.
Acceptance criteria:

- Given a logged-in user clicks 'Log Out', when the action is confirmed, then their session is terminated and they are redirected to the login screen.
- Given a user has logged out, when they try to access a protected page using the browser back button, then they are redirected to the login screen instead of seeing cached content.
- Given a user is mid-solve in Interactive Mode when they log out, when they log back in, then their in-progress solve is not resumed but their previously saved history remains intact.
- Given a logout request is sent, when it completes, then any saved session token or cookie is invalidated server-side, not just cleared from the browser.

7.3 Non-Functional Requirements

- Color detection shall be accurate under standard indoor lighting.
- Solution computation shall return a result within two seconds.
- The application shall respond to user interactions on mobile and desktop Chromium-based browsers without any extra installation.
- The 3D cube animation shall render smoothly at a consistent frame rate without visible lag during face rotations or camera orbit.
- The system shall validate the scanned cube state for correctness before attempting to solve, rejecting any illegal cube configuration.
- All user passwords shall be stored in hashed form; sensitive data shall not be exposed through any public-facing API endpoint.
- The database shall reliably persist all solve records and user history without data loss across sessions and devices.
- The user interface shall remain responsive and usable on screen widths as small as 360 pixels.
- The backend API shall return meaningful error messages on invalid input so the frontend can inform the user without crashing.